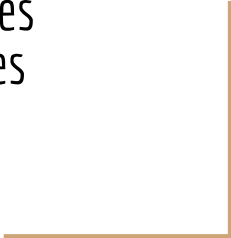


Também conhecido como Polimorfismo
Paramétrico

Templates em C++

Templates de Funções
Templates de Classes



Templates de Funções

Conceito de Templates

- Templates em C++ são mecanismos que permitem a criação de funções e classes genéricas, ou seja, que podem operar com diferentes tipos de dados sem que você precise escrever uma versão específica para cada tipo.
- Em vez de escrever várias funções ou classes com o mesmo código, mas para tipos diferentes (como `int`, `double`, `char`, etc.), você escreve um único "molde" — o template — que o compilador usa para gerar automaticamente o código correspondente para cada tipo utilizado no programa.

Templates de Funções

- Um **template de função** em C++ é um molde genérico para criar funções que funcionam com diferentes tipos de dados, sem precisar escrever uma nova função para cada tipo.
- Em vez de definir várias funções iguais que só diferem no tipo dos parâmetros, você define um template que o compilador usa para gerar automaticamente a função para o tipo que você usar no seu código.

Sintaxe para declarar

```
template <typename T> ou  
template <class T>
```

São equivalentes ... mesma coisa

Depois vem a declaração da função usando o tipo genérico T:

```
template <typename T>  
T nomeDaFuncao(T parametro) {  
    // implementação  
}
```

O que eu estou dizendo ?

```
template <typename T>
T nomeDaFuncao(T parametro) {
    // implementação
}
```

Estou dizendo que essa função deve funcionar para um tipo **T** que não existe em lugar nenhum... não é uma classe e nem um tipo primitivo.

Mas claro ... o tipo deve ser compatível como que você fizer dentro da função....

Por exemplo, se o corpo da função usa o operador `>` com o tipo **T**, o tipo passado deve suportar o operador `>`.

O que eu estou dizendo ?

```
template <typename T>
```

```
T max(T a, T b) {
```

```
    return (a > b) ? a : b;
```

```
    // T precisa ter operator>
```

```
}
```

Se **a** maior que **b**,
retorne **a**, senão
retorne **b**.

Estou dizendo que essa função deve funcionar para um tipo **T** que não existe em lugar nenhum... não é uma classe e nem um tipo primitivo.

Se chamar **max(3, 5)**, funciona, pois inteiros suportam **>**

Se chamar **max(obj1, obj2)** para um tipo **obj** que não tenha **operator>**, vai dar erro na compilação.

Funciona e não funciona com a função anterior. ..

```
// Classe sem sobrecarga
class Pessoa {
public:
    string nome;
    Pessoa(string n) : nome(n) {}
};
```

```
// Classe com sobrecarga do >
class Numero {
public:
    int valor;
    Numero(int v) : valor(v) {}
    bool operator>(const Numero& outro) const {
        return valor > outro.valor;
    }
};
```

Funciona e não funciona com a função anterior. ..

```
int main() {
    // Chamadas que funcionam:
    cout << "max(10, 20)          = " << max(10, 20)          << endl; // int
    cout << "max(3.14, 2.71)      = " << max(3.14, 2.71)      << endl; // double
    cout << "max('a', 'z')        = " << max('a', 'z')        << endl; // char
    cout << "max(string, string) = " << max(string("Ana"), string("Zoe")) << endl; // std::string
    cout << "max(Numero(5), Numero(9)).valor = " << max(Numero(5), Numero(9)).valor << endl;

    // Chamadas que NÃO funcionam

    Pessoa p1("João"), p2("Maria");
    max(p1, p2); // ERRO: Pessoa não tem operator>

    return 0;
}
```

Outro exemplo

- Quero uma única função que consiga imprimir arrays de qualquer tipo ... (ou pelo menos da maioria dos tipos)

```
#include <iostream>
```

```
using std::cout;
```

```
using std::endl;
```

```
template <class T>
```

```
void printArray(const T* inicioArray, const int count)
```

```
{
```

```
    for (int i = 0; i < count; i++)
```

```
        cout << inicioArray[i] << " ";
```

```
    cout << endl; // pula uma linha
```

```
} // end function printArray
```

```
int main()
```

```
{
```

```
    const int aCount = 5;
```

```
    const int bCount = 7;
```

```
    const int cCount = 6;
```

```
    int a[aCount] = { 1, 2, 3, 4, 5 };
```

```
    double b[bCount] = { 1.1, 2.2, 3.3, 4.4, 5.5, 6.6};
```

```
    char c[cCount] = "HELLO"; // 6th position for null
```

```
    cout << "Array a contains:" << endl;
```

```
    printArray(a, aCount); // integer
```

```
    cout << "Array b contains:" << endl;
```

```
    printArray(b, bCount);
```

```
    cout << "Array c contains:" << endl;
```

```
    printArray(c, cCount); // character
```

Exercício - menor elemento

Implemente uma **função template** chamada `menorElemento` que receba um array de qualquer tipo e o número de elementos desse array, e retorne o **menor elemento** presente nele.

Você deve usar um **template de função**, de forma que a mesma função funcione para arrays de `int`, `double` e `char`.

Teste sua função no `main()` com:

Um array de inteiros

Um array de números reais (`double`)

Um array de caracteres (`char`)

Exercício - menor elemento ++

Você deve reaproveitar a função template `menorElemento`, que recebe um array de qualquer tipo e retorna o menor elemento, mas agora o array será de objetos de uma classe chamada `Pessoa`.

Cada objeto da classe `Pessoa` deve armazenar: um nome (string) e uma idade (int)

Faça uma nova versão da função **`menorElemento()`** de forma que ela consiga retornar a pessoa mais jovem que estiver no array de `Pessoa`

Template de Classes

Templates de Classes

- Templates de classes servem para **criar classes genéricas**, ou seja, classes que funcionam com **qualquer tipo de dado**, sem precisar reescrevê-las para cada tipo.
- **Objetivo** → Evitar **duplicação de código** ao criar **estruturas de dados ou classes utilitárias** que funcionam com diferentes tipos.
- Exemplo
 - Suponha que você queira criar uma classe Caixa que armazene um valor qualquer.
 - Sem templates, você precisaria duplicar o código para CaixaInt, CaixaDouble, CaixaString, etc.
 - Com template de classe, você faz algo assim →

Templates de Classes

```
template <typename T>

class Caixa {
private:
    T valor;

public:
    Caixa(T v) : valor(v) {}
    T getValor() const { return valor; }
};
```

Como declarar objetos ?

Detalhe importante !
Tem que falar para a classe
quem é o tipo T

```
Caixa<int> c1(42);
```

```
Caixa<double> c2(3.14);
```

```
Caixa<std::string> c3("texto");
```

Exemplo

Imagine que estamos criando um sistema e precisamos de uma classe genérica que possa armazenar **qualquer tipo de objeto**, como Produto, Cliente, Funcionario, etc.

Devemos então criar uma classe template chamada Repositorio<T>, que armazena um vetor de objetos do tipo T

```
#include <iostream> e #include <vector> e #include <string>
```

```
class Produto {  
private:  
    std::string nome;  
    double preco;  
  
public:  
    Produto(std::string n, double p) : nome(n), preco(p) {}  
    void exibir() const {  
        std::cout << "Produto: " << nome << " - R$" << preco << std::endl;  
    }  
};
```

```
// Template de classe genérica para armazenar objetos
```

```
template <typename T>  
class Repositorio {  
private:  
    std::vector<T> dados;  
  
public:  
    void adicionar(const T& item) {  
        dados.push_back(item);  
    }  
  
    void listar() const {  
        for (const T& item : dados) {  
            item.exibir(); // T precisa ter um método exibir()  
        }  
    }  
};
```

```
int main() {  
    Produto p1("Notebook", 3500.0);  
    Produto p2("Mouse", 80.0);  
  
    Repositorio<Produto> repoProdutos;  
    repoProdutos.adicionar(p1);  
    repoProdutos.adicionar(p2);  
  
    std::cout << "Lista de produtos:" <<  
std::endl;  
    repoProdutos.listar();  
  
    return 0;  
}
```

No Exemplo anterior, poderia com qq tipo ?

- Sim, mas depende das operações/métodos que serão usados da classe
- A Classe Repositório pode ser instanciada com qualquer outro tipo de Classe que exista (Cliente, Produto, Carro, Pedido, Veiculo, etc)
- Mas, deve-se cuidar para invocar métodos que existam nessas classes
- Por exemplo:

```
int main() {  
    // Repositório de Clientes  
    Repositorio<Cliente> repoClientes;  
    repoClientes.adicionar(Cliente("Alice"));  
  
    // Repositório de Pedidos  
    Repositorio<Pedido> repoPedidos;  
    repoPedidos.adicionar(Pedido(202));  
  
    // Repositório de Carros  
    Repositorio<Carro> repoCarros;  
    repoCarros.adicionar(Carro("Toyota Corolla"));  
  
    // Listagem  
    cout << "--- Clientes ---" << endl;  
    repoClientes.listar();  
  
    cout << "\n--- Pedidos ---" << endl;  
    repoPedidos.listar();  
  
    cout << "\n--- Carros ---" << endl;  
    repoCarros.listar();  
  
    return 0;  
}
```

Só funciona se o método `exibir()` estiver definido dentro de cada classe T

Exercício

- Crie uma classe Comparador que serve para comparar vários tipos de objetos
- A classe deve receber dois objetos do tipo T e ser capaz de compará-los com o operador >
- A class então
 - deve ter dois atributos do tipo T
 - construtor que recebe os dois atributos do tipo T
 - método maior() que retorna o maior dos dois objetos
 - método exibirMaior() que imprime o maior objeto com "cout"
- No Main
 - teste com tipos primitivos int e double
 - Crie uma classe Aluno com atributos nome e nota
 - Use o Comparador<Aluno> para descobrir qual aluno tem a maior nota
 -